

Abstrakt

Einleitung und Projektziele

Ziel des Projekts war die Entwicklung einer nativen Windows-Desktop-Anwendung, die lokale Bild- und Videodateien semantisch durchsuchbar macht. Im Gegensatz zu klassischen Ansätzen, die auf Dateinamen oder manuellen Metadaten basieren, sollte die Anwendung den Inhalt von Medien mithilfe vortrainierter multimodaler KI-Modelle verstehen und vergleichen. Der Kundenwert liegt in der Kombination aus Zeitersparnis und lokalem Datenschutz: Nutzer sollen große Mediensammlungen per Texteingabe oder Referenzbild durchsuchen können, ohne ihre Daten an Cloud-Dienste zu übertragen. Als technische Grundlage wurde BLIP (Bootstrapping Language-Image Pre-training) gewählt, das sowohl Bildvektorisierung als auch automatische Bildbeschreibungen ermöglicht.

Entwicklungsverlauf und unerwartete Probleme

Die Entwicklung erfolgte unter WSL (Windows Subsystem for Linux), da die Entwicklungsumgebung bereits installiert war. Die eigentliche Implementierung verlief unter Linux problemlos und zügig: Python, PySide6 und die BLIP-Modelle ließen sich erstaunlich einfach integrieren, genau wie die Anbindung der HuggingFace-Modelle. Die eigentlichen Schwierigkeiten traten erst beim Build-Prozess auf. Da die Anwendung als native Windows-EXE ausgeliefert werden sollte, wurde PyInstaller über einen GitHub-Actions-Workflow auf einem Windows-Runner kompiliert. Diese Entscheidung hatte einen erheblichen Mehraufwand zur Folge: Fehler, die unter Linux nicht auftraten, zeigten sich erst nach dem Kompilierungsvorgang auf GitHub – beispielsweise Sonderzeichen in Pfaden, die unter Windows nicht zulässig sind, fehlende Schreibrechte für den Modellordner sowie inkompatible CUDA-Versionen zwischen PyTorch und den installierten Treibern. Jeder dieser Fehler erforderte einen neuen Kompilierungsdurchlauf, was aufgrund der Buildzeit von mehreren Minuten pro Durchlauf auf GitHub zeitaufwendig war. Im Rückblick wäre es deutlich effizienter gewesen, die Entwicklungsumgebung von Anfang an nativ unter Windows aufzusetzen, um Plattformunterschiede frühzeitig zu erkennen.

Technische Herausforderungen: Textsuche und Modality Gap

Eine der zentralen Herausforderungen stellte der Vergleich von Texteingaben mit Bildvektoren dar. Der ursprüngliche Ansatz, den Nutzer-Prompt direkt als Vektor mit den Bildvektoren zu vergleichen, lieferte in der Praxis teilweise unsinnige Ergebnisse: Selbst wenn der KI-Prompt eines Bildes den exakt gesuchten Gegenstand oder das gesuchte Tier erkannt hatte, war der Vektor eines Bildes ohne den gesuchten Inhalt manchmal dennoch näher am Suchvektor, also der Modality Gap.

Die implementierte Lösung umgeht dieses Problem, indem der Nutzer-Prompt nicht mit dem Bildvektor, sondern per Schlagwortabgleich mit der generierten Caption verglichen wird (siehe `calculate_strict_keyword_score` in `engine/ai_worker.py`). Für jedes Wort des bereinigten Prompts wird geprüft, ob es in den Wörtern der Caption vorkommt; der resultierende Anteil bestimmt den Score. Diese Lösung funktionierte in der Praxis recht zuverlässig, hat jedoch Grenzen bei abstrakten Anfragen oder Synonymen, die nicht im generierten Caption-Vokabular auftreten.

Positive Erkenntnisse

Trotz der genannten Probleme zeigte das Projekt, dass die Integration vortrainierter KI-Modelle in Desktop-Anwendungen mit modernen Python-Bibliotheken technisch einfach umzusetzen ist. Die gewählte Schichtenarchitektur ermöglichte es, die Suchlogik mehrfach grundlegend zu überarbeiten, ohne die Benutzeroberfläche anpassen zu müssen. Ebenso war die Auslagerung rechenintensiver Aufgaben in QThread-Instanzen eine gute Lösung, da die GUI auch bei längerer Modellverarbeitung jederzeit reaktionsfähig bleibt.

Was ich beim nächsten Projekt anders machen würde

Bei Projekten, deren Zielplattform Windows ist, sollte die Entwicklungsumgebung ebenfalls von Anfang an unter Windows eingerichtet werden, auch wenn Linux im Umgang oft einfacher ist. Die Kosten später auftretender Plattformfehler, insbesondere wenn jeder Fix einen vollständigen CI-Build erfordert, überwiegen den anfänglichen Mehraufwand vermutlich bei Weitem. Technisch würde ich früher eine lokale Windows-Testumgebung einrichten und den Build-Prozess bereits während der Entwicklungsphase regelmäßig ausführen, anstatt ihn erst am Ende zu validieren.

Fazit

Die Anwendung erfüllt die gesetzten Kernziele: Lokale semantische Suche in Bild- und Videodateien ist möglich, alle Verarbeitungsschritte verbleiben auf dem Gerät des Nutzers, und die Benutzeroberfläche bleibt auch bei laufender KI-Verarbeitung stabil. Das Projekt hat verdeutlicht, dass der Mehrwert moderner KI-Modelle nicht nur in ihrer Leistungsfähigkeit liegt, sondern auch darin, wie sie in eine nutzbare und erklärbare Anwendung eingebettet werden. Genau diese Integration, zwischen Modellausgabe, Benutzeroberfläche und verständlichem Ergebnis, war ja das eigentliche Ziel dieses Projekts.